Computer Arch.

Lecture 11-14: Assembly Programming

From Prof. Onur Mutlu

Agenda for Today & Next Few Lectures

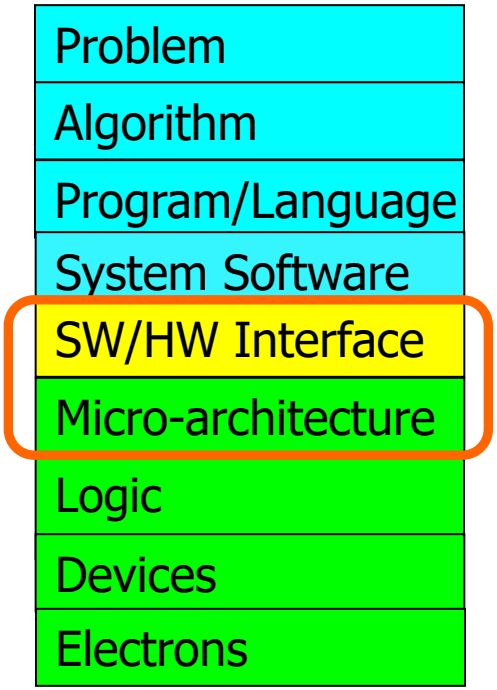
- The von Neumann model
- LC-3: An example of von Neumann machine

- LC-3 and MIPS Instruction Set Architectures

- LC-3 and MIPS assembly and programming

- Introduction to microarchitecture and single-cycle microarchitecture

- Multi-cycle microarchitecture



What Will We Learn?

- Assembly Programming
  - Programming constructs
  - Debugging
  - Conditional statements and loops in MIPS assembly
  - Arrays in MIPS assembly
  - Function calls
    - The stack

OPERATE INSTRUCTIONS

NOT in LC-3

- NOT assembly and machine code

LC-3 assembly

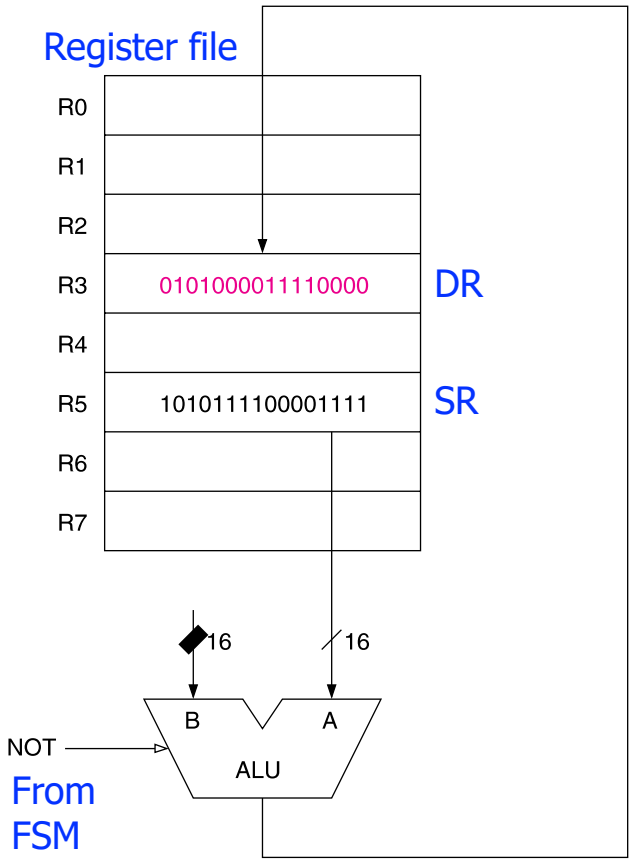
```
NOT   R3, R5
```

Field Values

OP	DR	SR	
9	3	5	1 1 1 1 1 1

Machine Code

OP	DR	SR	
1 0 0 1	0 1 1	0 0 1	1 1 1 1 1 1
15 12	11 9	8 6	5 0



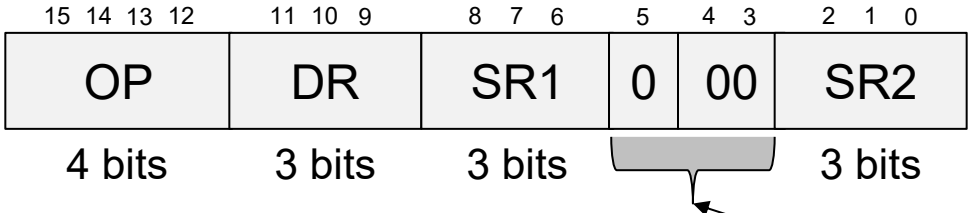
There is no NOT in MIPS. How is it implemented?

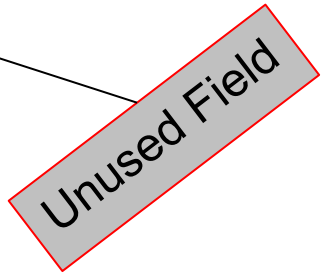
Operate Instructions

- We are already familiar with LC-3's ADD and AND with register mode (R-type in MIPS)
- Now let us see the versions with one literal (i.e., immediate) operand
- We will use Subtraction as an example
  - How is it implemented in LC-3 and MIPS?

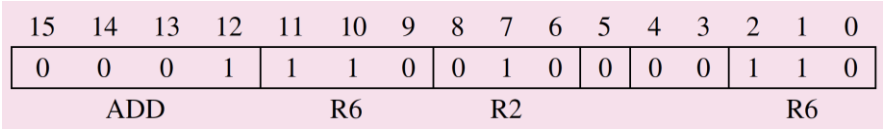
Recall: LC-3 Operate Instruction Format

- LC-3 Operate Instruction Format (Register OP Register)



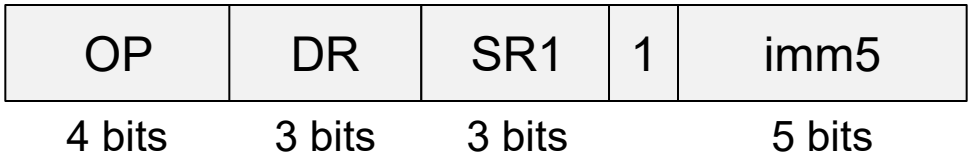


  - OP = opcode (what the instruction does)
    - E.g., ADD = 0001
      - Semantics: DR ← SR1 + SR2
    - E.g., AND = 0101
      - Semantics: DR ← SR1 AND SR2
  - SR1, SR2 = source registers
  - DR = destination register



Operate Instr. with one Literal in LC-3

- ADD and AND



  - OP = operation
    - E.g., ADD = 0001 (same OP as the register-mode ADD)
      - DR ← SR1 + sign-extend(imm5)
    - E.g., AND = 0101 (same OP as the register-mode AND)
      - DR ← SR1 AND sign-extend(imm5)
  - SR1 = source register
  - DR = destination register
  - imm5 = Literal or immediate (sign-extend to 16 bits)

ADD with one Literal in LC-3

- ADD assembly and machine code

LC-3 assembly

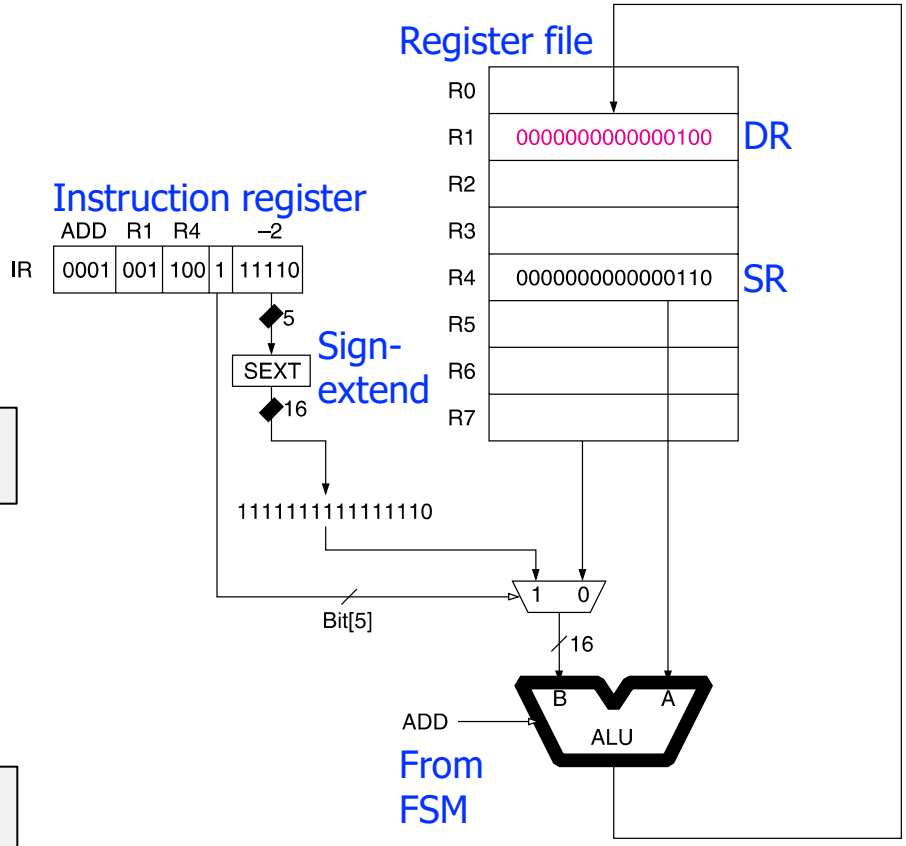
```
ADD R1, R4, #-2
```

Field Values

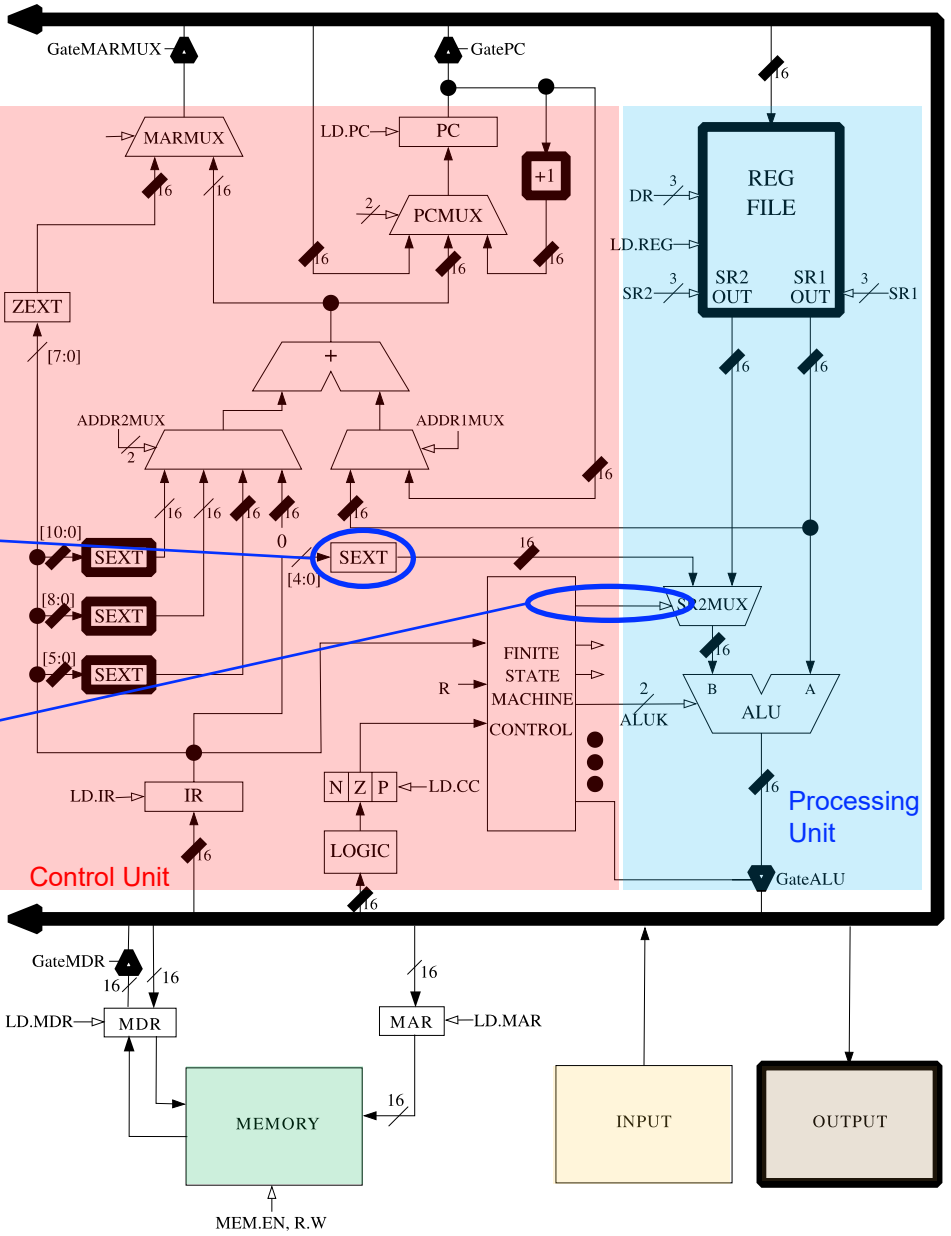
OP	DR	SR		imm5
1	1	4	1	-2

Machine Code

OP	DR	SR		imm5
0 0 0 1	0 0 1	1 0 0	1	1 1 1 1 0
15 12	11 9	8 6	5	4 0



ADD with one Literal in LC-3 Data Path

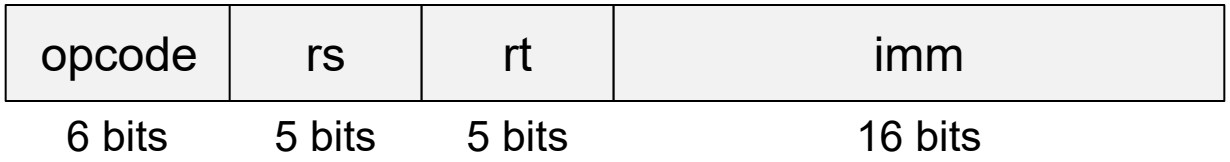


Sign extension (Operand)

Select Immediate or Register (as the 2nd input to instruction)

Instructions with one Literal in MIPS

- I-type MIPS Instructions
  - 2 register operands and immediate
- Some operate and data movement instructions



  - opcode = operation
  - rs = source register
  - rt =
    - destination register in some instructions (e.g., `addi`, `lw`)
    - source register in others (e.g., `sw`)
  - imm = Literal or immediate

ADD with one Literal in MIPS

- Add immediate

MIPS assembly

```
addi $s0, $s1, 5
```

Dollar sign $ is used primarily to designate general-purpose registers.

Field Values

op	rs	rt	imm
8	17	16	5

rt ← rs + sign-extend(imm)

Machine Code

op	rs	rt	imm
001000	10001	10010	0000 0000 0000 0101

0x22300005

Subtraction in MIPS vs. LC-3

- MIPS assembly

High-level code

```
a = b + c - d;
```

MIPS assembly

```
add  $t0, $s0, $s1
sub  $s3, $t0, $s2
```

- LC-3 assembly

High-level code

```
a = b + c - d;
```

LC-3 assembly

```
ADD  R2, R0, R1
NOT  R4, R3
ADD  R5, R4, #1
ADD  R6, R2, R5
```

2's complement of R3

- Tradeoff in LC-3
  - More instructions
  - But, simpler control logic

Subtract Immediate

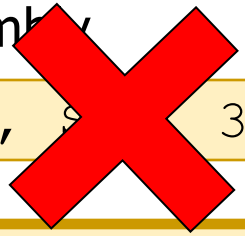
- MIPS assembly

High-level code

```
a = b - 3;
```

MIPS assembly

```
subi $s1, $s0, 3
```



Is subi necessary in MIPS?

MIPS assembly

```
addi $s1, $s0, -3
```

- LC-3

High-level code

```
a = b - 3;
```

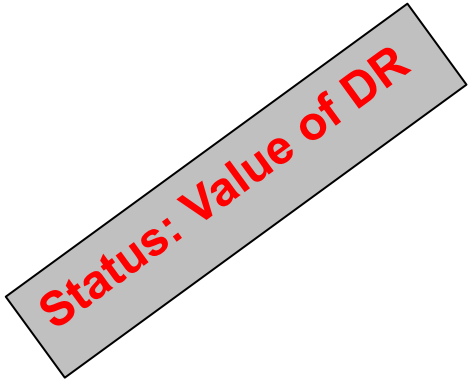
LC-3 assembly

```
ADD  R1, R0, #-3
```

Conditional Control Flow (Conditional Branching)

Condition Codes in LC-3

- Each time one GPR (R0-R7) is written, three single-bit registers are updated
- Each of these condition codes are either set (set to 1) or cleared (set to 0)
  - If the written value is negative
    - N is set, Z and P are cleared
  - If the written value is zero
    - Z is set, N and P are cleared
  - If the written value is positive
    - P is set, N and Z are cleared

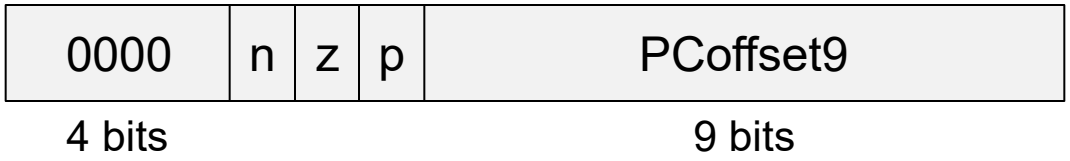


- x86 and SPARC are examples of ISAs that use condition codes

Conditional Branches in LC-3

- BRz (Branch if Zero)

```
BRz   PCoffset9
```



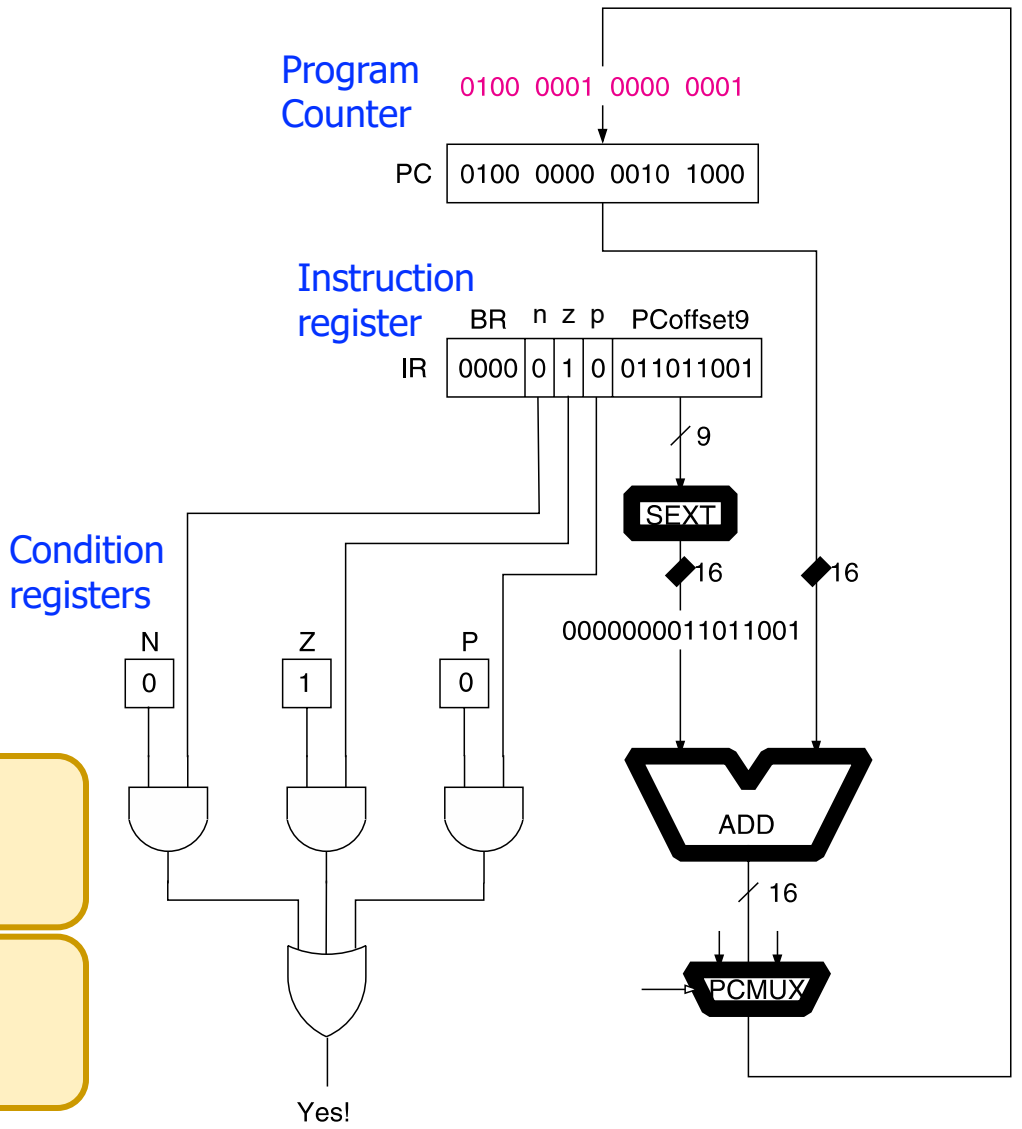
  - n, z, p = which condition code is tested (N, Z, and/or P)
    - n, z, p: instruction bits to identify the condition codes to be tested
    - N, Z, P: values of the corresponding condition codes
  - PCoffset9 = immediate or constant value
  - if ((n AND N) OR (p AND P) OR (z AND Z))
    - then PC ← PC† + sign-extend(PCoffset9)
  - Variations: BRn, BRz, BRp, BRzp, BRnp, BRnz, BRnzp

† This is the incremented PC

Conditional Branches in LC-3

- BRz

```
BRz   0x0D9
```



What if n = z = p = 1?*
(i.e., BRnzp)

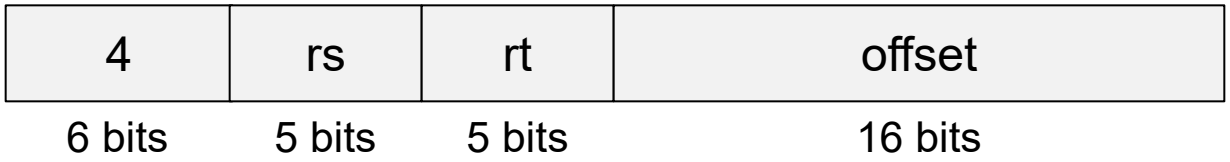
And what if n = z = p = 0?

*n, z, p are the instruction bits to identify the condition codes to be tested

Conditional Branches in MIPS

- beq (Branch if Equal)

```
beq  $s0, $s1, offset
```



  - 4 = opcode
  - rs, rt = source registers
  - offset = immediate or constant value
  - if rs == rt
    - then PC ← PC† + sign-extend(offset) * 4
  - Variations: beq, bne, blez, bgtz

† This is the incremented PC

Branch If Equal in MIPS and LC-3

MIPS assembly

```
beq  $s0, $s1, offset
```

LC-3 assembly

```
NOT  R2, R1
ADD  R3, R2, #1
ADD  R4, R3, R0
BRz  offset
```

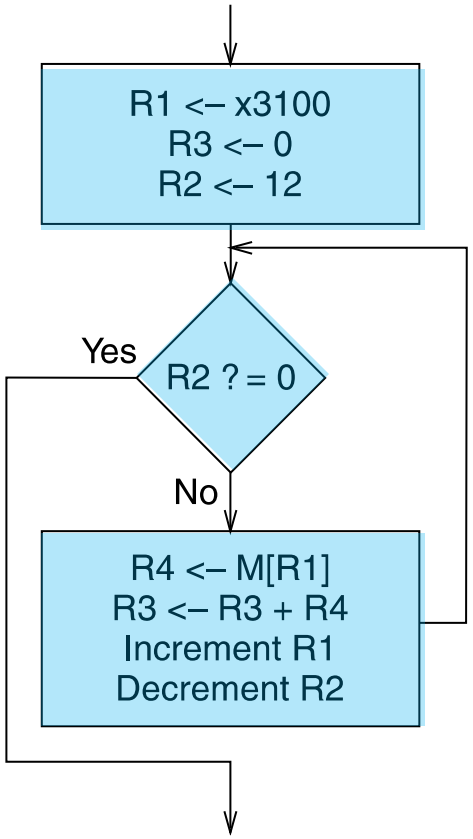
Subtract (R0 - R1)

If R0 = R1

- This is an example of tradeoff in the instruction set
  - The same functionality requires more instructions in LC-3
  - But, the control logic requires more complexity in MIPS

An Algorithm for Adding Integers

- We want to write a program that adds 12 integers
  - They are stored in addresses 0x3100 to 0x310B
  - Let us take a look at the flowchart of the algorithm



R1: initial address of integers

R3: final result of addition

R2: number of integers left to be added

Check if R2 becomes 0 (done with all integers?)

Load integer in R4

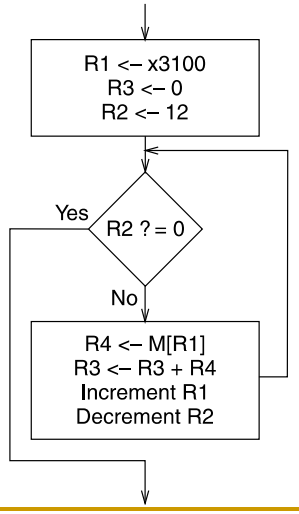
Accumulate integer value in R3

Increment address R1
Decrement R2

A Program for Adding Integers in LC-3

- We use conditional branch instructions to create a loop

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	LEA	1	1	0	0	0	1	0x00FF									R1 = PC† + 0x00FF = 3100 // load address
x3001	AND	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	R3 = 0 // reset register
x3002	AND	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	R2 = 0 // reset register
x3003	ADD	0	0	1	0	1	0	0	1	0	1	0	1	1	0	0	R2 = R2 + 12 // initialize counter
x3004	BR	0	0	0	0	z	0	5									BRz (PC † + 5) = BRz 0x300A // check condition
x3005	LDR	1	1	0	1	0	0	0	0	1	0	0	0	0	0	0	R4 = M[R1 + 0] // load value
x3006	ADD	0	0	1	0	1	1	0	1	1	0	0	0	1	0	0	R3 = R3 + R4 // accumulate
x3007	ADD	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	R1 = R1 + 1 // increment address
x3008	ADD	0	0	1	0	1	0	0	1	0	1	-1					R2 = R2 – 1 // decrement counter
x3009	BR	0	0	0	n	z	p	-6									BRnzp (PC † – 6) = BRnzp 0x3004 // jump



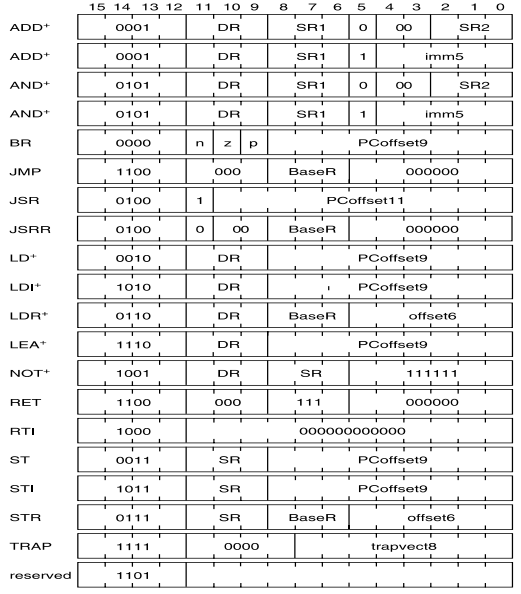


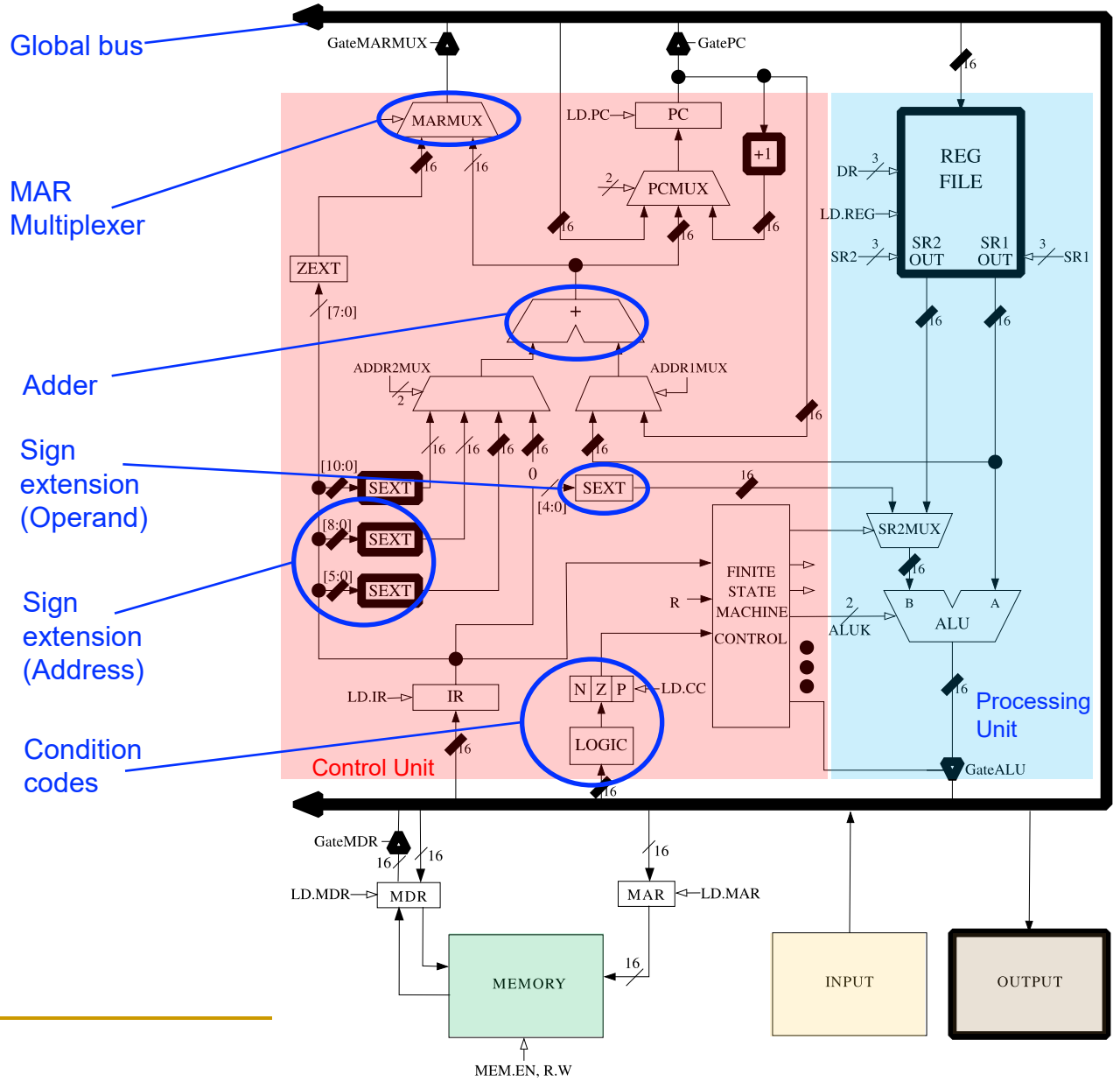
Figure 5.3 Formats of the entire LC-3 instruction set. NOTE: † indicates instructions that modify condition codes

Bit 5 to differentiate the two ADD instructions

The LC-3 Data Path Revisited

The LC-3 Data Path

We highlight some data path components used in the execution of the instructions in the previous slides (not shown in the simplified data path)

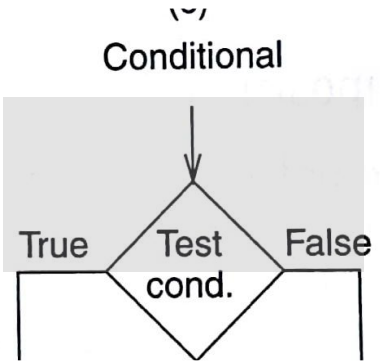


(Assembly) Programming

Programming Constructs

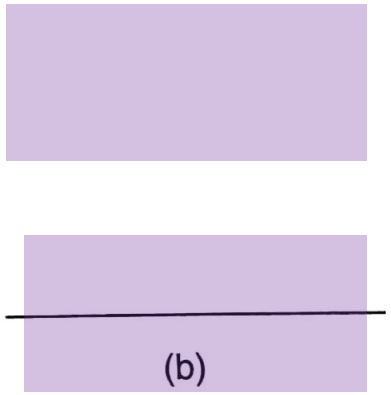
- Programming requires dividing a task, i.e., a unit of work into smaller units of work
- The goal is to replace the units of work with programming constructs that represent that part of the task
- There are three basic programming constructs
  - Sequential construct
  - Conditional construct
  - Iterative construct

In programming, a "construct" broadly refers to a fundamental building block or mechanism provided by a programming language to implement specific concepts or control program flow



Sequential Construct

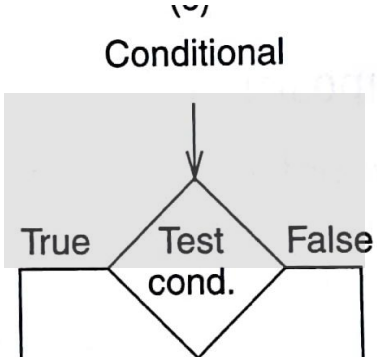
- The sequential construct is used if the designated task can be broken down into two subtasks, one following the other

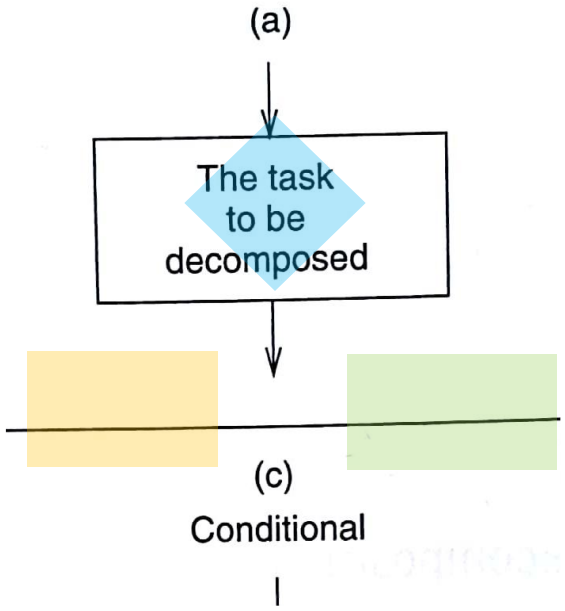


(b)
Sequential

Conditional Construct

- The conditional construct is used if the designated task consists of doing one of two subtasks, but not both



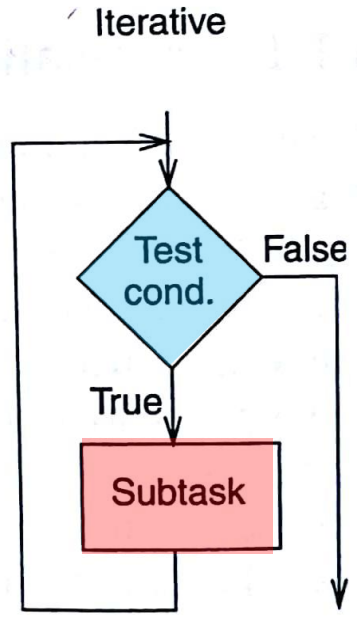


Is the condition "true" or "false"?

  - Either subtask may be "do nothing"
  - After the correct subtask is completed, the program moves onward
- E.g., if-else statement, switch-case statement

Iterative Construct

- The iterative construct is used if the designated task consists of doing a subtask a number of times, but only as long as some condition is true



Is the condition still "true"?

- E.g., for loop, while loop, do-while loop

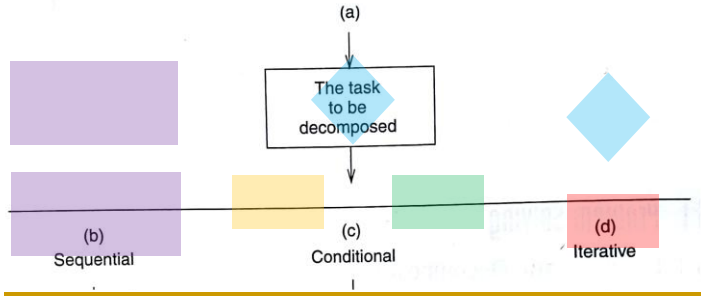
Constructs in an Example Program

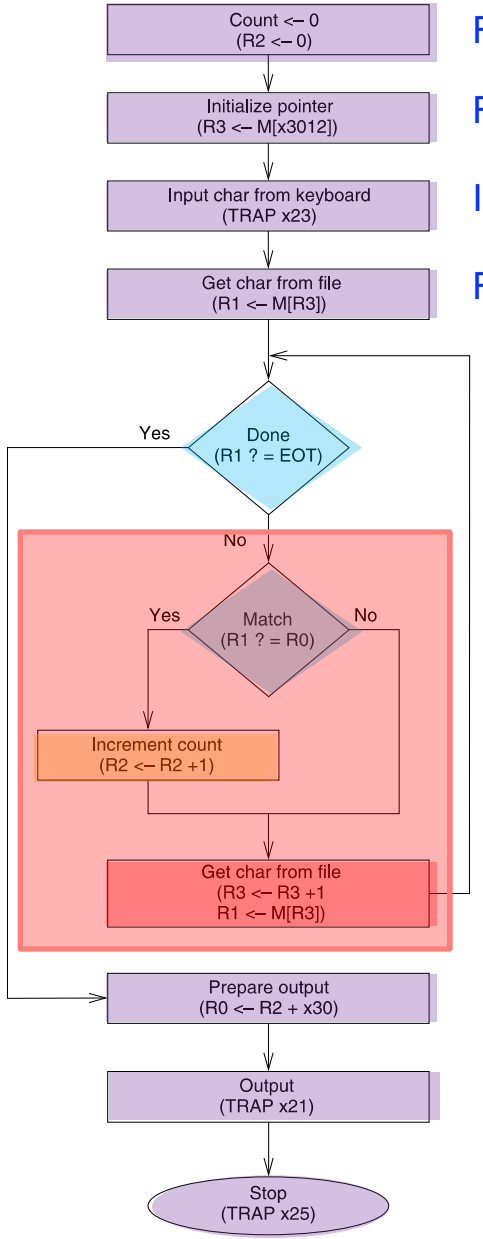
- Let us see how to use the programming constructs in an example program
- The example program counts the number of occurrences of a character in a text file
- It uses sequential, conditional, and iterative constructs
- We will see how to write conditional and iterative constructs with conditional branches

Counting Occurrences of a Character

- We want to write a program that counts the occurrences of a character in a file
  - Get character-to-search from the keyboard (TRAP instr.)
  - The file finishes with the character EOT (End Of Text)
    - That is called a sentinel
    - In this example, EOT = 4
  - Output result to the monitor (TRAP instr.)

Programming constructs





R2: counter

R3: initial address

Input char

Read char from file

Check if end of file

Is it the searched char?

Increment R2

Increment address
Read char from file

Move output to R0

Output counter

Halt the program

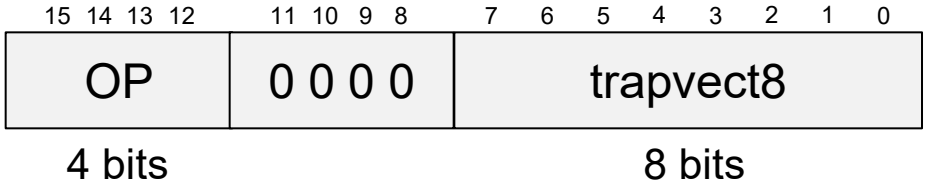
TRAP Instruction

- TRAP invokes an OS service call

LC-3 assembly

```
TRAP 0x23;
```

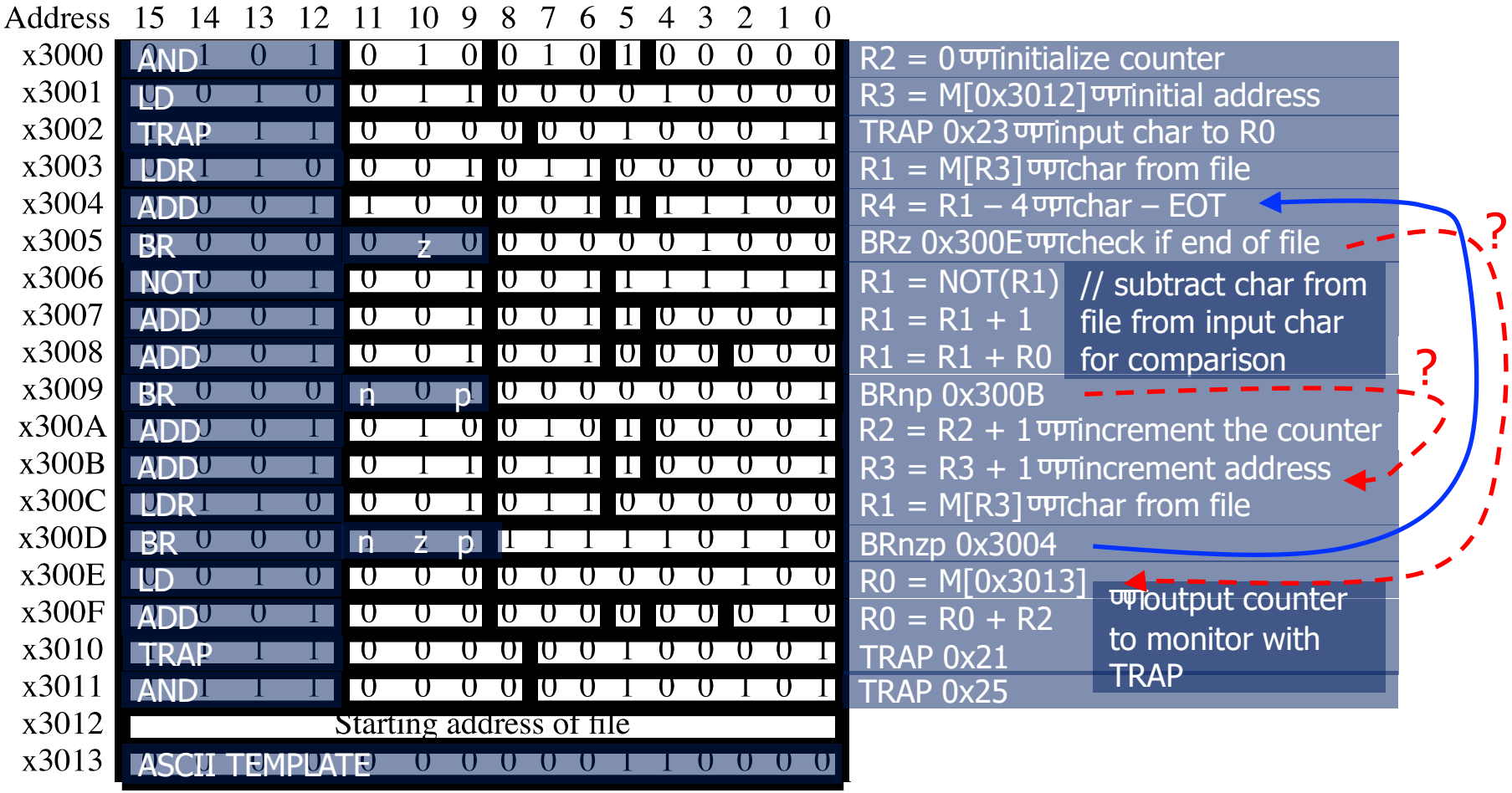
Machine Code



- OP = 1111
- trapvect8 = service call
  - 0x23 = Input a character from the keyboard
  - 0x21 = Output a character to the monitor
  - 0x25 = Halt the program

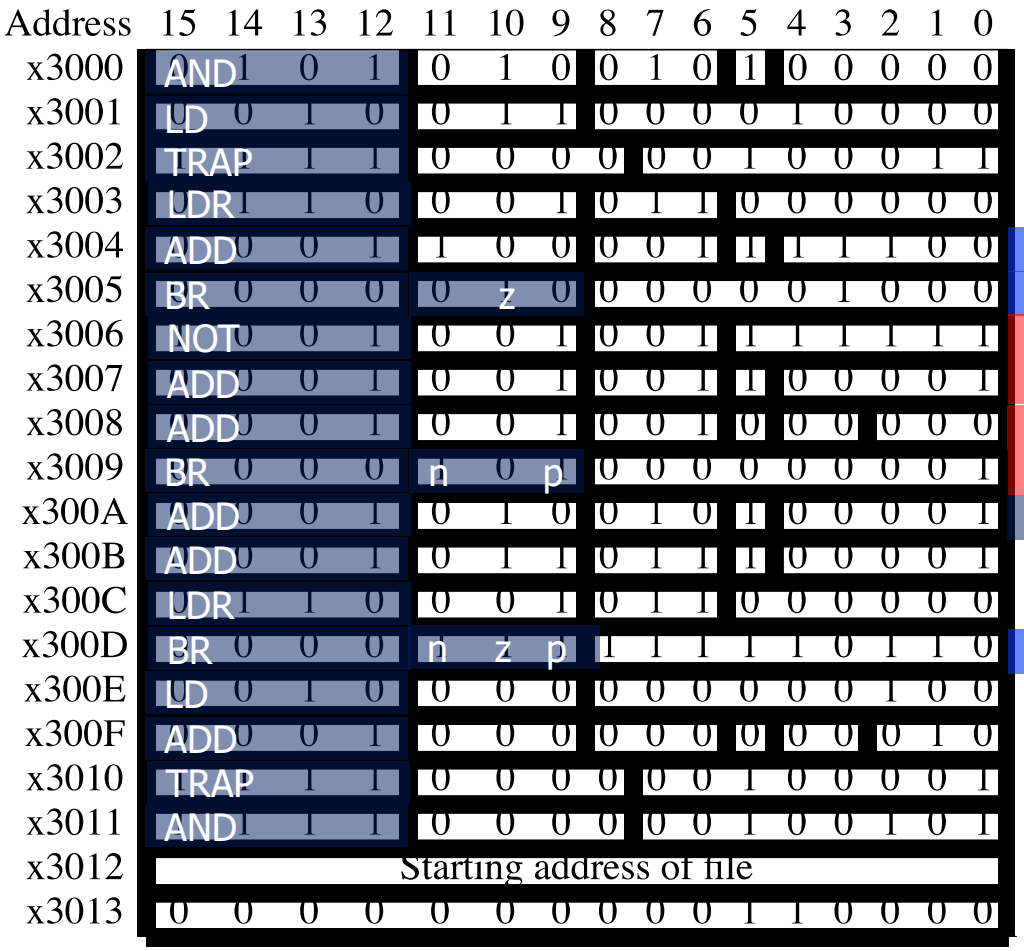
Counting Occurrences of a Char in LC-3

- We use conditional branch instructions to create loops and if statements



Programming Constructs in LC-3

- Let us do some reverse engineering to identify conditional constructs and iterative constructs



Debugging

Debugging

- Debugging is the process of removing errors in programs
- It consists of tracing the program, i.e., keeping track of the sequence of instructions that have been executed and the results produced by each instruction
- A useful technique is to partition the program into parts, often referred to as modules, and examine the results computed in each module
- High-level language (e.g., C programming language) debuggers: dbx, gdb, Visual Studio debugger
- Machine code debugging: Elementary interactive debugging operations

Interactive Debugging

- When debugging interactively, it is important to be able to
  - 1. Deposit values in memory and in registers, in order to test the execution of a part of a program in isolation
  - 2. Execute instruction sequences in a program by using
    - RUN command: execute until HALT instruction or a breakpoint
    - STEP N command: execute a fixed number (N) of instructions
  - 3. Stop execution when desired
    - SET BREAKPOINT command: stop execution at a specific instruction in a program
  - 4. Examine what is in memory and registers at any point in the program

Example: Multiplying in LC-3 (Buggy)

- A program is necessary to multiply, since LC-3 does not have multiply instruction
  - The following program multiplies R4 and R5
  - Initially, R4 = 10 and R5 = 3
  - The program produces 40. What went wrong?
  - It is useful to annotate each instruction

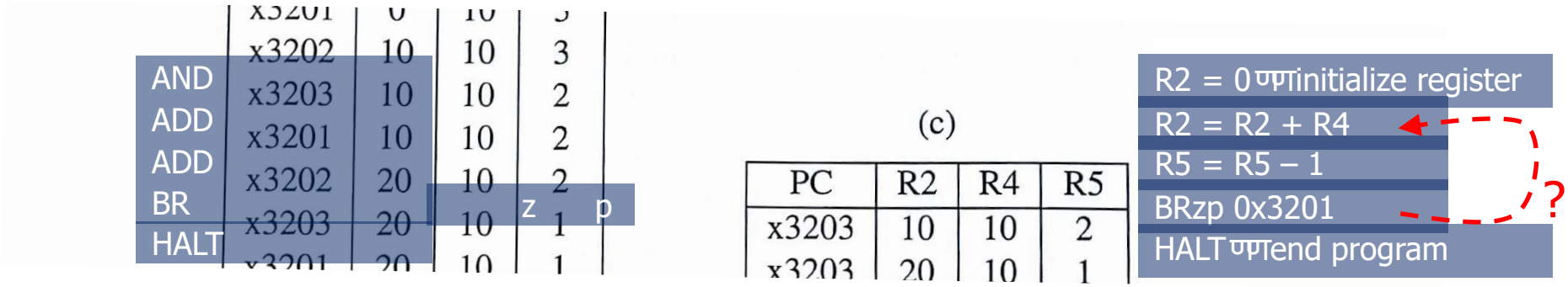
	x3201	0	10	3	
AND	x3202	10	10	3	
ADD	x3203	10	10	2	
ADD	x3201	10	10	2	
BR	x3202	20	10	2	z p
HALT	x3203	20	10	1	
	x3201	20	10	1	

(c)

PC	R2	R4	R5
x3203	10	10	2
x3203	20	10	1

R2 = 0 initialize register
R2 = R2 + R4
R5 = R5 – 1
BRzp 0x3201
HALT end program
?

Debugging the Multiply Program



- We examine the contents of all registers after the execution of each instruction

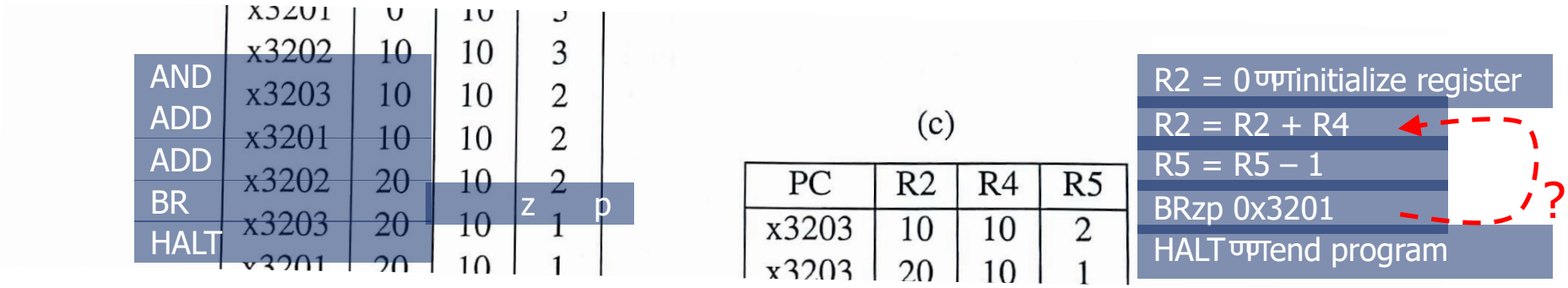
PC	R2	R4	R5	
x3201	0	10	3	
x3202	10	10	3	
x3203	10	10	2	
x3201	10	10	2	
x3202	20	10	2	
x3203	20	10	1	
x3201	20	10	1	
x3202	30	10	1	← Correct result
x3203	30	10	0	← BR should not be taken if R5 = 0
x3201	30	10	0	
x3202	40	10	0	
x3203	40	10	−1	
x3204	40	10	−1	
	40	10	−1	

The branch condition codes were set wrong. The conditional branch should only be taken if R5 is positive

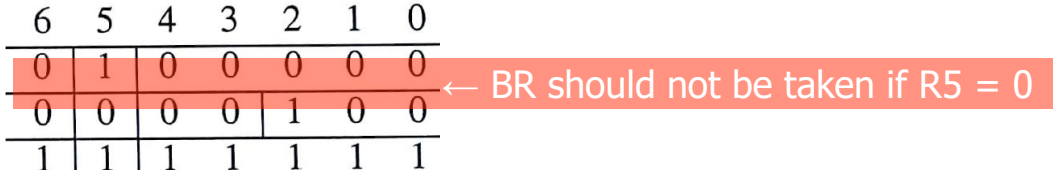
Correct instruction:

```
BRp  #-3      // BRp 0x3201
```

Easier Debugging with Breakpoints



- We could use a breakpoint to save some work
- Setting a breakpoint in 0x3203 (BR) allows us to examine the results of each iteration of the loop



One last question:
Does this program work if the initial value of R5 is 0?

A good test should also consider the corner cases,
i.e., unusual values that the programmer might fail to consider

Conditional Statements and Loops in MIPS Assembly

If Statement

- In MIPS, we create conditional constructs with conditional branches (e.g., beq, bne…)

High-level code

```
if (i == j)
  f = g + h;

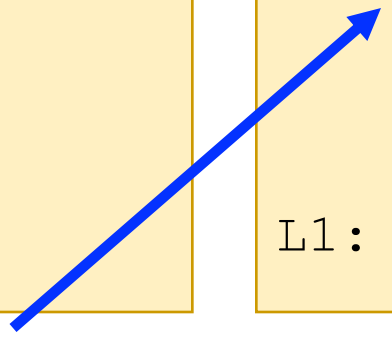
f = f - i;
```

MIPS assembly

```
# $s0 = f, $s1 = g
# $s2 = h
# $s3 = i, $s4 = j

    bne $s3, $s4, L1
    add $s0, $s1, $s2

L1: sub $s0, $s0, $s3
```



Branch not equal

Compares two values (`$s3=i, $s4=j`) and jumps if they are different

If-Else Statement

- We use the unconditional branch (i.e., j) to skip the "else" subtask if the "if" subtask is the correct one

High-level code

```
if (i == j)
  f = g + h;
else
  f = f - i;
```

MIPS assembly

```
# $s0 = f, $s1 = g,
# $s2 = h
# $s3 = i, $s4 = j

      bne $s3, $s4, L1
      add $s0, $s1, $s2
      j   done
L1:   sub $s0, $s0, $s3
done:
```

1. Compare two values ($s3=i, $s4=j) and, if they are different, jump to L1, to execute the "else" subtask

2. Jump to done, after executing the "if" subtask

While Loop

- As in LC-3, the conditional branch (i.e., beq) checks the condition and the unconditional branch (i.e., j) jumps to the beginning of the loop

High-level code

```
// determines the power
// of 2 equal to 128
int pow = 1;
int x   = 0;

while (pow != 128) {
  pow = pow * 2;
  x = x + 1;
}
```

MIPS assembly

```
# $s0 = pow, $s1 = x

        addi $s0, $0, 1
        add  $s1, $0, $0
        addi $t0, $0, 128
while:  beq  $s0, $t0, done
        sll  $s0, $s0, 1
        addi $s1, $s1, 1
        j    while
done:
```

1. Conditional branch to check if the condition still holds

2. Unconditional branch to the beginning of the loop

For Loop

- The implementation of the "for" loop is similar to the "while" loop

High-level code

```
// add the numbers from 0 to 9

int sum = 0;
int i;
for (i = 0; i != 10; i = i+1)
{
  sum = sum + i;
}
```

MIPS assembly

```
# $s0 = i, $s1 = sum
        addi $s1, $0, 0
        add  $s0, $0, $0
        addi $t0, $0, 10
for:    beq  $s0, $t0, done
        add  $s1, $s1, $s0
        addi $s0, $s0, 1
        j    for
done:
```

1. Conditional branch to check if the condition still holds

2. Unconditional branch to the beginning of the loop

For Loop Using SLT

- We use slt (i.e., set less than) for the "less than" comparison

High-level code

```
// add the powers of 2 from 1
// to 100
int sum = 0;
int i;

for (i = 1; i < 101; i = i*2)
{
  sum = sum + i;
}
```

MIPS assembly

```
# $s0 = i, $s1 = sum

        addi $s1, $0, 0    Initialize sum
        addi $s0, $0, 1    and i
        addi $t0, $0, 101
loop:   slt  $t1, $s0, $t0
        beq  $t1, $0, done
        add  $s1, $s1, $s0
        sll  $s0, $s0, 1
        j    loop
done:
```

Set less than

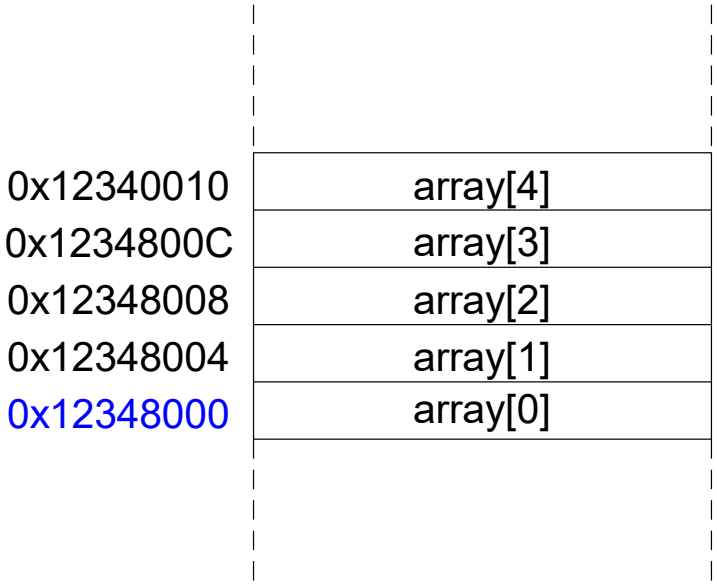
`$t1 = $s0 < $t0 ? 1:0`

Shift left logical

Arrays in MIPS

Arrays

- Accessing an array requires loading the base address into a register



- In MIPS, this is something we cannot do with one single immediate operation
- Load upper immediate + OR immediate

```
lui   $s0, 0x1234
ori   $s0, $s0, 0x8000
```

Arrays: Code Example

- We first load the base address of the array into a register (e.g., $s0) using lui and ori

High-level code

```
int array[5];

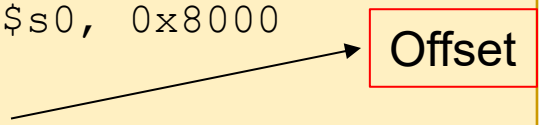

array[0] = array[0] * 2;


array[1] = array[1] * 2;
```

MIPS assembly

```
# array base address = $s0
# Initialize $s0 to 0x12348000
lui   $s0, 0x1234
ori   $s0, $s0, 0x8000

lw    $t1, 0($s0)
sll   $t1, $t1, 1
sw    $t1, 0($s0)
lw    $t1, 4($s0)
sll   $t1, $t1, 1
sw    $t1, 4($s0)
```



Function Calls

Function Calls

- Why functions (i.e., procedures)?
  - Frequently accessed code
  - Make a program more modular and readable
- Functions have arguments and return value

- Caller: calling function
  - main()
- Callee: called function
  - sum()

```
void main()
{
  int y;
  y = sum(42, 7);
  ...
}

int sum(int a, int b)
{
  return (a + b);
}
```

Function Calls: Conventions

- Conventions
  - Caller
    - passes arguments
    - jumps to callee
  - Callee
    - performs the procedure
    - returns the result to caller
    - returns to the point of call
    - must not overwrite registers or memory needed by the caller

Function Calls in MIPS and LC-3

- Conventions in MIPS and LC-3
  - Call procedure
    - MIPS: Jump and link (jal)
    - LC-3: Jump to Subroutine (JSR, JSRR)
  - Return from procedure
    - MIPS: Jump register (jr)
    - LC-3: Return from Subroutine (RET)
  - Argument values
    - MIPS: $a0 - $a3
  - Return value
    - MIPS: $v0

Function Calls: Simple Example

High-level code

```
int main() {
  simple();
  a = b + c;
}

void simple() {
  return;
}
```

MIPS assembly

```
0x00400200 main: jal simple
0x00400204       add $s0,$s1,$s2



...
0x00401020 simple: jr $ra
```

- jal jumps to simple() and saves PC+4 in the return address register ($ra)
  - $ra = 0x00400204
  - In LC-3, JSR(R) put the return address in R7
- jr $ra jumps to address in $ra (LC-3 uses RET instruction)

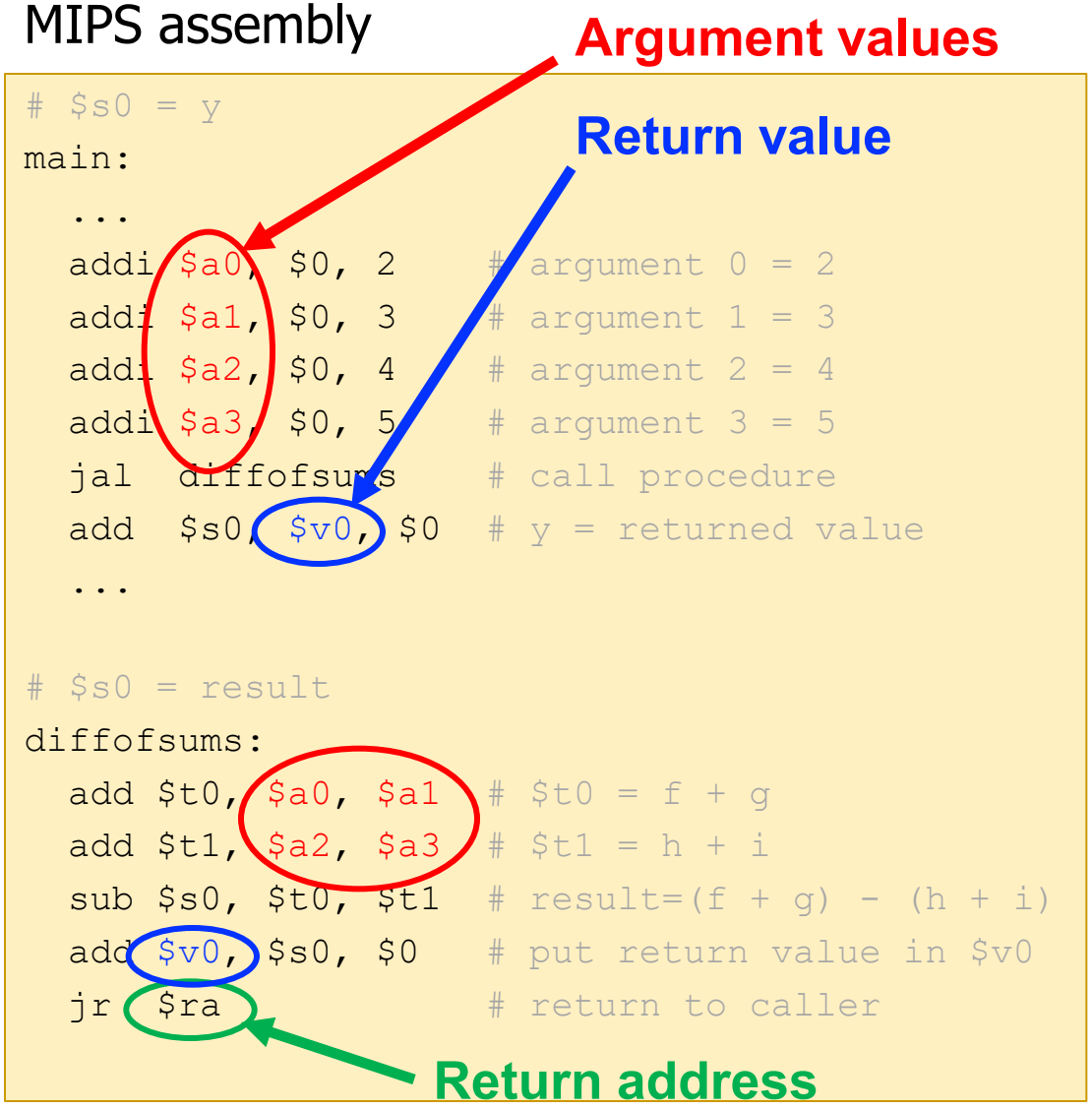
Function Calls: Code Example

High-level code

```
int main()
{
  int y;
  ...
  // 4 arguments
  y = diffofsums(2, 3, 4, 5);
  ...
}

int diffofsums(int f, int g,
  int h, int i)
{
  int result;
  result = (f + g) - (h + i);
  // return value
  return result;
}
```

MIPS assembly



Function Calls: Need for the Stack

MIPS assembly

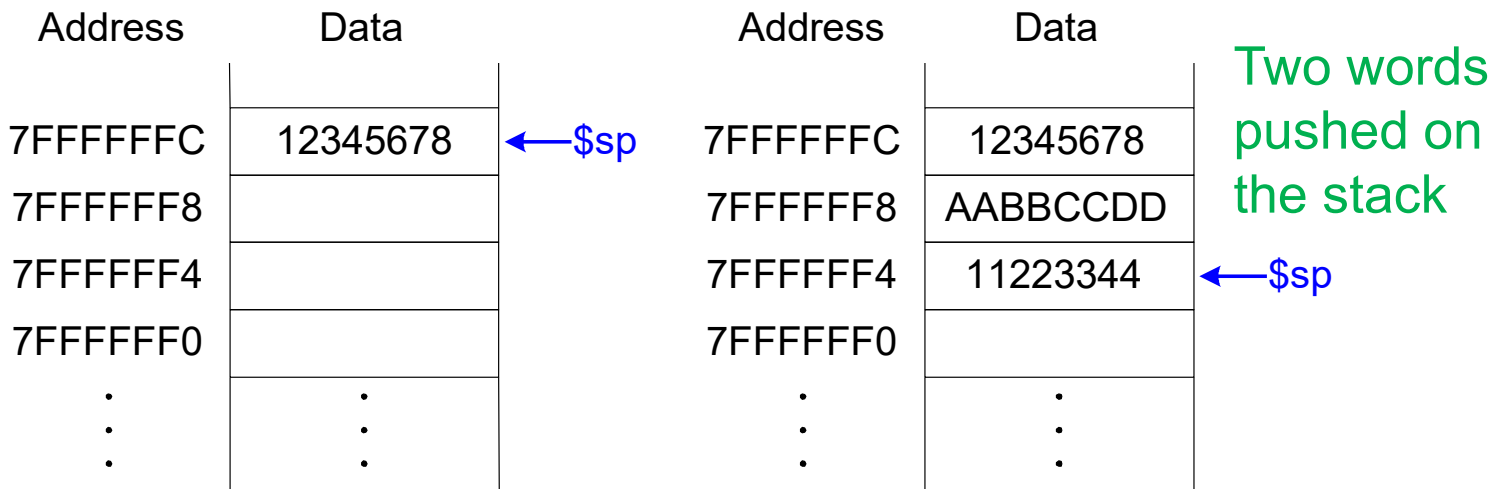
```
diffofsums:
  add $t0, $a0, $a1   # $t0 = f + g
  add $t1, $a2, $a3   # $t1 = h + i
  sub $s0, $t0, $t1   # result=(f + g) - (h + i)
  add $v0, $s0, $0    # put return value in $v0
  jr  $ra             # return to caller
```

- What if the main function was using some of those registers?
  - $t0, $t1, $s0
- They could be overwritten by the function
- We can use the stack to temporarily store registers

The Stack

- The stack is a memory area used to save local variables
- It is a Last-In-First-Out (LIFO) queue
- The stack pointer ($sp) points to the top of the stack
  - It grows down in MIPS

The stack in MIPS serves as a temporary storage area in memory, primarily used for managing function calls and local data.



The Stack: Code Example

MIPS assembly

```
diffofsums:
  addi $sp, $sp, -12  # allocate space on stack to store 3 registers
  sw   $s0, 8($sp)    # save $s0 on stack
  sw   $t0, 4($sp)    # save $t0 on stack
  sw   $t1, 0($sp)    # save $t1 on stack
  add $t0, $a0, $a1   # $t0 = f + g
  add $t1, $a2, $a3   # $t1 = h + i
  sub $s0, $t0, $t1   # result=(f + g) - (h + i)
  add $v0, $s0, $0    # put return value in $v0
  lw   $t1, 0($sp)    # restore $t1 from stack
  lw   $t0, 4($sp)    # restore $t0 from stack
  lw   $s0, 8($sp)    # restore $s0 from stack
  addi $sp, $sp, 12   # deallocate stack space
  jr  $ra             # return to caller
```

- Saving and restoring all registers requires a lot of effort
- In MIPS, there is a convention about temporary registers (i.e., $t0-$t9): There is no need to save them
  - Programmers can use them for temporary/partial results

MIPS Stack: Register Saving Convention

MIPS assembly

```
diffofsums:
  addi $sp, $sp, -4    # allocate space on stack to store 1 register
  sw   $s0, 0($sp)     # save $s0 on stack


  add $t0, $a0, $a1    # $t0 = f + g
  add $t1, $a2, $a3    # $t1 = h + i
  sub $s0, $t0, $t1    # result=(f + g) - (h + i)
  add $v0, $s0, $0     # put return value in $v0


  lw   $s0, 0($sp)     # restore $s0 from stack
  addi $sp, $sp, 4     # deallocate stack space
  jr  $ra              # return to caller
```

In MIPS assembly language, $0 (or $zero) refers to a special-purpose register that is hardwired to the value zero.

Any attempt to write a different value to register $0 will be ignored, and it will always return 0 when read.

- Temporary registers $t0-$t9 are nonpreserved registers. They are not saved, thus, they can be overwritten by the function
- Registers $s0-$s7 are preserved (saved; callee-saved) registers

Lecture Summary

- Assembly Programming
  - Programming constructs
  - Debugging
  - Conditional statements and loops in MIPS assembly
  - Arrays in MIPS assembly
  - Function calls
    - The stack